



Homework 11

Please complete the following problems. You must submit the following for each problem: a) the answer you are asked to return and b) Python code that can be run to verify your work.

1. Assessing Assembly Quality with N50 and N75

The goal of genome sequencing is to create contigs that are as long as possible. Thus, after fragment assembly, it is important to generate statistics quantifying how well-assembled our contigs are.

First and foremost, we demand a measure of what percentage of the assembled genome is made up of long contigs. Our first question is then: if we select contigs from our collection, how long do the contigs need to be to cover 50% of the genome?

PROBLEM

Given a collection of DNA strings representing contigs, we use the N statistic NXX (where XX ranges from 01 to 99) to represent the maximum positive integer L such that the total number of nucleotides of all contigs having length $\geq L$ is at least XX% of the sum of contig lengths. The most common such statistic is N50, although N75 is also worth mentioning.

Given: A file containing a collection of at most 1000 DNA strings (whose combined length does not exceed 50 kb).

Return: N50 and N75 for this collection of strings.

Sample Dataset

```
GATTACA
TACTACTAC
ATTGAT
GAAGA
```

Sample Output

```
7 6
```

Hint: For an explanation of the results obtained in the sample above, contigs of length at least 7 total $7 + 9 = 16$ bp, which is more than 50% of the total 27). Contigs of length at least 8 total only 9 bp (less than 50%).

Contigs of length at least 6 total $6 + 7 + 9 = 22$ bp, which is more than 75% of all base pairs. Contigs of length at least 7 total only 16 bp (less than 75%).

2. Transcript Assembly and Quantification

When it comes to RNA-seq analysis, there are hundreds of programs, each with unique characteristics and applications, used by researchers worldwide. Even when focused on the popular task of determining differentially expressed genes (DEGs), how to select the appropriate workflow is a challenge for many, especially those new to the field of bioinformatics. If computer resources are abundant, it is recommended to run multiple pipelines and compare results to get the most reliable DEG set. However, it takes effort to figure out how to run all the different tools and make sure the data from one pipeline program is compatible with downstream programs.

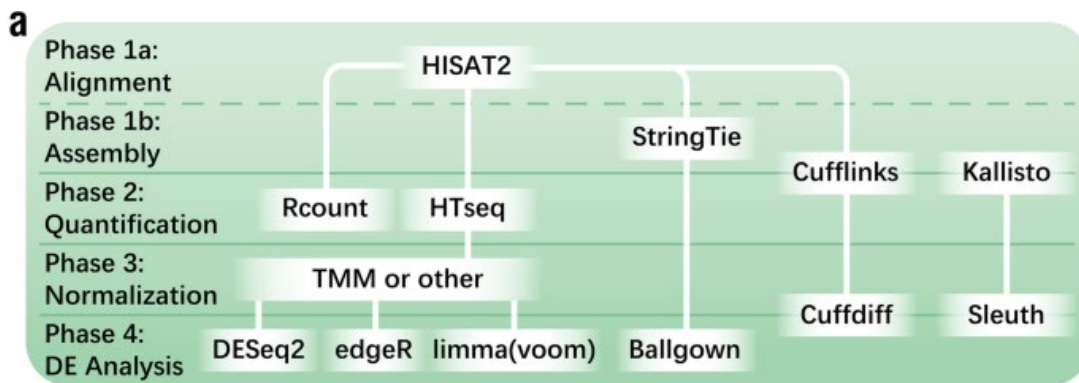


Figure 1: Overview of popular RNA-seq data analysis pipelines ([source](#)).

One popular RNA-seq data analysis pipeline for identifying DEGs is HISAT2-StringTie-Ballgown. HISAT2 is a fast, sensitive spliced alignment program designed to align RNA-seq reads to a reference genome sequence. The program takes the sequencing reads in FASTQ format and the reference sequence in FASTA format and outputs an aligned reads file in BAM format.

StringTie is a fast and highly efficient program that assembles RNA-seq alignments into transcripts and quantifies these transcripts, essentially telling you how many reads were aligned to each gene/transcript. HISAT2 assembly just aligns the sequencing reads to the reference genome, it does not define or count genes/transcripts, this is the job of StringTie which uses units of fragments per kilobase of transcript per million mapped reads (FPKM) to describe the number of reads per transcript. Research shows that this quantification step has a bigger impact on RNA-seq results than alignment. StringTie takes the BAM file produced by HISAT2 and generates a GTF file identifying the assembled transcripts that were quantified and tabular files containing the quantification data in FPKM units for each assembled transcript.

Finally, if you are interested in DEGs, then you need an algorithm to take your transcript counts and determine if there is a significant difference in gene expression between your experimental groups.

PROBLEM

For this problem and the next, you are going to analyze data from the NCBI BioProject “Redesign of an *Escherichia coli* Nissle treatment for phenylketonuria using insulated genomic landing pads and genetic circuits to reduce burden” ([link](#)). In this project, researchers are engineering an *E. coli* strain to work better as a human therapeutic drug (the original strain was discussed in ENBC353). Importantly, although you are analyzing a single sample from this project, assessing differential expression between experimental groups requires multiple samples per group to be analyzed for appropriate statistical significance.



Given: The following files:

1. Raw RNA sequencing reads in FASTQ format.
2. Reference genome sequence in FASTA format.
3. Reference genome gene and transcript annotations in GFF3 format.

Return: Five files in tabular format and compatible with the Ballgown program that were produced by the HISAT2-StringTie pipeline.

Note: instead of code, please submit screenshots showing successful runs of HISAT2 and StringTie. The screenshots can be of your Galaxy and/or the command line.

Hint: The easiest way to run HISAT2 is via Galaxy. For the downstream files to be compatible with Ballgown, you must select “Use a genome from history” for the “Source for the reference genome” setting, selecting the provided *E. coli* reference genome FASTA file. The sequencing reads in the FASTQ file represent a single-end library. To ensure maximum compatibility of the resulting HISAT2 BAM file with the downstream StringTie program, you should open “Advanced Options,” select “Specify spliced alignment options,” and click “Report alignments tailored for transcript assemblers including StringTie” for the “Transcriptome assembly reporting” option.

There are multiple ways to run StringTie, but the two easiest methods are to run the program in Galaxy or Docker. In Galaxy, select “Short reads” for the “Input options” setting and choose the HISAT2 BAM file as the input. To obtain the required Ballgown-compatible tabular files, you must select the “Use reference GTF/GFF3” file for the “Use a reference file to guide assembly?” setting. StringTie can perform *de novo* assembly of RNA transcripts, essentially defining the RNA molecules to be counted based on the sequencing read data. By using the provided annotation file in GFF3 format, you are telling StringTie to count the transcripts defined by the GFF3 file and avoid *de novo* assembly. When you make this change in Galaxy, you open the “Output files for differential expression?” setting. Select the “Ballgown” option to instruct StringTie to generate five files containing quantification data in the Ballgown format. Ballgown needs all five of these files to perform the differential expression analysis.

With Docker Desktop already installed, it is simple to install and run StringTie. Use the Docker pull command **`docker pull ctomlins/stringtie2`** to download a [StringTie2 image](#). (There are other StringTie image options available in the Docker Hub, but the ctomlins image was verified.) The **`docker run -it ctomlins/stringtie2`** command will open a Docker container running StringTie. Change to the **`stringtie-2.2.1.Linux_x86_64`** directory to run the program. This is the folder in which you need to import the HISAT2 BAM file and reference genome annotation GFF3 file. When your input files are ready, you can run StringTie with the following command:

```
stringtie -o output_gtf_filename.gtf -G reference_genome_annotation_filename.gff -e -B
HISAT2_output_filename.bam
```

The -B setting will tell StringTie to create the Ballgown-compatible output files. This will be five different files all with the .ctab file extension. The StringTie manual is [here](#) if you want to learn more about running the program from the command line and the different options available.



3. Performing Differential Expression Analysis on the Command Line

Again, there are many available differential expression (DE) analysis tools. As part of the HISAT2-StringTie-Ballgown pipeline introduced in the “Transcript Assembly and Quantification” problem, you are going to use Ballgown to identify differentially expressed genes (DEGs) or genes in which there is a statistically significant difference in gene expression between experimental groups based on the RNA sequencing read counts.

Ballgown is an R software package designed to facilitate flexible differential expression analysis of RNA-seq data. Despite being one of the most popular DE analysis tools, Ballgown is not installed on public Galaxy servers, although it is available in the [Galaxy Tool Shed](#) for installation on local Galaxy instances. To run Ballgown on the command line, you need to download and install the latest version of R and then install [Bioconductor](#). Ballgown guides and a reference manual can be found [here](#), [here](#), and [here](#), respectively.

You are going to use Ballgown to determine the number of DEGs in a provided dataset for only two experimental groups with two samples in each group. However, Ballgown is a much more powerful program that can perform DE analysis on more samples and more groups, customize the DE analysis with different statistical models, and create an array of useful plots describing the results.

PROBLEM

Ballgown requires its input files to be in a very specific format. Transcript quantification data for a single sample is split among five different input files with the .ctab extension. You created these five files for a single sample in the “Transcript Assembly and Quantification” problem. However, to run DE analysis, you need this set of five files for each sample of the experimental groups being statistically compared. Again, DE analysis requires multiple samples per experimental group like any proper scientific experiment.

For this problem, you are going to analyze data from the NCBI BioProject “Redesign of an Escherichia coli Nissle treatment for phenylketonuria using insulated genomic landing pads and genetic circuits to reduce burden” ([link](#)). The two experimental groups to be compared are a wild-type *E. coli* strain, and *E. coli* strain engineered to express high levels of recombinant proteins from the genome, not from a plasmid. The authors used RNA sequencing to assess the impact of their genome edits on the transcriptome. Ideally, the data would show that the wild-type and engineered strains have highly similar transcriptomes, signaling that their genome changes do not cause any unanticipated negative effects on *E. coli*.

Given: Two sets of StringTie-generated transcript quantification tables in tabular format for each experimental group (four sets of tables total, 20 .ctab files total).

Return: The number of DEGs between the two experimental groups ($p < 0.05$).

Note: instead of code, please submit a screenshot of the command line showing a successful run of the required Ballgown commands.

Hint: Ballgown is an unexpectedly tricky program to learn but having some familiarity with the R language will be helpful. [Bioconductor](#) must be installed to install Ballgown. It is also recommended to install core Bioconductor packages using the `BiocManager::install(c("GenomicFeatures", "AnnotationDbi"))` command. If Bioconductor is ready, then the installation commands in this Ballgown [guide](#) should work.



Loading your RNA-seq data into R to be read by Ballgown is the trickiest part. There are various ways to do this, but the simplest of them is described below. First, you need to run the `library(ballgown)` command to attach the ballgown library. To automatically find the important `extdata` folder associated with your installed Ballgown package and link it to the `data_directory` variable, run `data_directory = system.file('extdata', package='ballgown')`. To examine the directory so that you can find the folder on your local machine, simply type and run `data_directory`.

You want to find the `extdata` folder on your local machine because this is where you need to move the given problem data, or if you were running the entire analysis on your own datasets, where you need to move your StringTie-generated data. The goal is to load the transcript quantification data into a ballgown object. You can do this with the command `bg = ballgown(dataDir=data_directory, samplePattern='sample', meas='all')`. Importantly, this command only works when the data is loaded into the `extdata` folder in a very specific format for which Ballgown already has a template created when you explore the folder. The folder structure looks like this:

```
extdata/  
  sample01/  
    e2t.ctab  
    e_data.ctab  
    i2t.ctab  
    i_data.ctab  
    t_data.ctab  
  sample02/  
    e2t.ctab  
    e_data.ctab  
    i2t.ctab  
    i_data.ctab  
    t_data.ctab  
  ...  
  sample20/  
    e2t.ctab  
    e_data.ctab  
    i2t.ctab  
    i_data.ctab  
    t_data.ctab
```

Since you are given four samples, you want to create four folders (sample01, sample02, sample03, sample04) that each contain a set of five output files. You can check out a Ballgown guide (e.g., this [one](#)) for more details on each output file (the `t_data.ctab` file is the most important as it contains expression measurements or counts for each transcript assembled by StringTie).

With the output files in the correct folder, the `bg = ballgown(dataDir=data_directory, samplePattern='sample', meas='all')` command will create a ballgown object `bg`. Ballgown objects. Type and run `bg` to confirm that a ballgown object was created. It should include data for about 4,500 transcripts for each of 4 samples. A ballgown object has six slots: `structure`, `expr`, `indexes`, `dirs`, `mergedDate`, and `meas`, of which you can read about on your own.

What the ballgown object does not include is definition of the experimental groups, which is critical for DE analysis. You assign a sample to an experimental group by creating an R data frame named `pData`. Use this command: `pData(bg) = data.frame(id=sampleNames(bg), group=c(1,0))`. This command creates the `pData` data frame containing a column of sample names from the ballgown object `bg` that you just created and assigns them to the experimental group defined in the column `group=c(0,0,1,1)`, which is simply a column of values 0, 0, 1, and 1. The zeroes and ones correspond to different experimental groups. You must make sure you know which data is in the sample01, sample02, etc. folders so that you can properly assign both wild-type samples to the same group (e.g., 0) and the test samples to the other group (e.g., 1). Typing and running `pData(bg)` will show you the `pData` data frame, which again should just be a table that assigns each sample to an experimental group.



Creating a ballgown object and defining the experimental groups with `pData` is required to use the `stattest` function. Run the following: `stat_results = stattest(bg, feature='transcript', meas='FPKM', covariate='group')`. This command performs the DE analysis/statistical test for the data in the `bg` ballgown object, specifically analyzing the transcript quantification data in FPKM units across groups as defined in `pData(bg)`. When you run this command, no results will show up on the screen. To check that the test was done correctly, you can check out the first few lines of the R variable containing the results with the `head(stat_results)` command. It should show you a list of transcripts and the p- and q-values associated with the statistical comparison between experimental groups made by the `stattest` function.

You are almost done. All that is left is to count the number of DEGs as asked for by the problem. The problem defines a DEG as a transcript associated with a p-value less than 0.05, a typical statistical significance level. Thus, DEGs are simply genes with expression levels that are different between the two experimental groups to a statistically significant degree. In other words, DEGs are RNA transcripts with statistically significant differences in aligned sequencing reads when comparing the wild-type and engineered *E. coli* cell lines. The `subset` function and command `sig_results = subset(stat_results, stat_results$pval<0.05)` will create an R value that only lists the differentially expressed transcripts with a p-value less than 0.05. Finally, you can write the list to a TSV file using the command `write.table(sig_results, "outputfilename.tsv", sep="\t", quote=FALSE, row.names=FALSE)`. Notably, the newly created TSV file does not go to the `extdata` folder but to the current R directory that you can identify with the `getwd()` command.